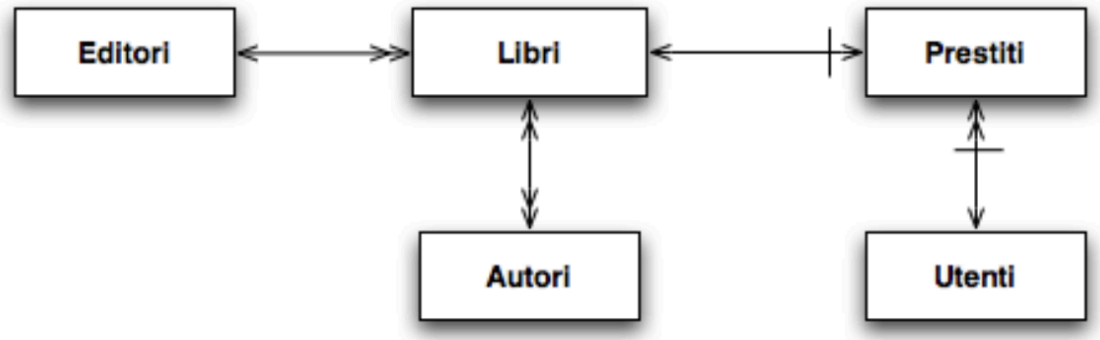
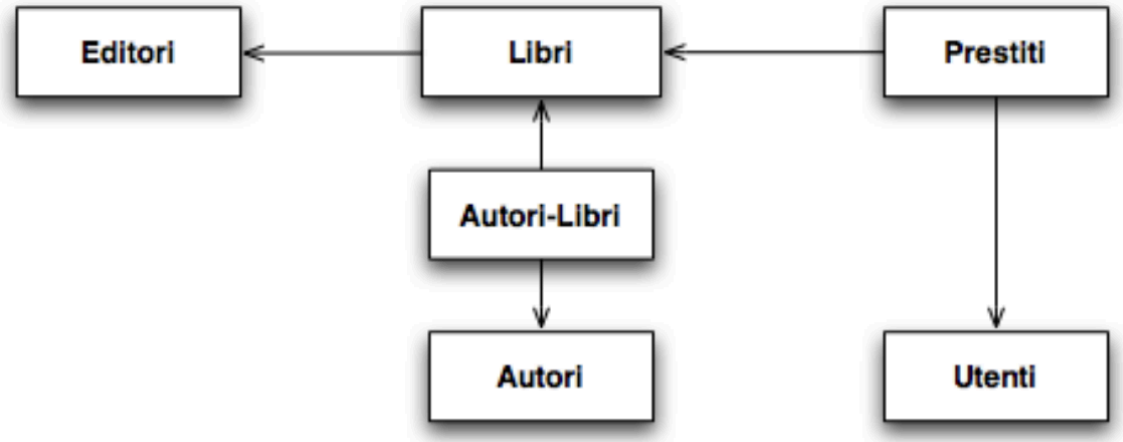
Trasformazione Schemi

Schema concettuale



Schema logico-relazionale



Progettazione Logica

Obiettivo

Tradurre lo **schema concettuale** (modello E/R o UML) in uno **schema logico-relazionale** che:

- rappresenti le **stesse informazioni** del concettuale;
- **minimizzi la ridondanza** (evitando duplicazioni inutili → anomalie di aggiornamento, cancellazione, inserimento);
- sia **comprensibile e coerente** → facile da mantenere e usare nelle applicazioni;
- tenga conto delle **prestazioni**, quindi non è solo "traduzione meccanica" ma anche **ottimizzazione**.

Osservazioni

- Non è una conversione 1:1: alcuni costrutti del modello concettuale non hanno un corrispettivo diretto nel modello logico (es. generalizzazioni, associazioni molti-a-molti, attributi multivalore).
- Serve **ristrutturare il modello concettuale** per arrivare a uno schema logico efficiente e implementabile in un DBMS relazionale.

Dati di Ingresso

1. **Schema concettuale** (diagramma E/R o UML, con entità, associazioni, attributi, cardinalità).
2. **Informazioni sul carico applicativo**, cioè:
   - volumi di dati attesi (dimensioni delle tabelle, crescita prevista);
   - tipi di operazioni più frequenti (letture, scritture, join, ricerche, aggiornamenti).
3. **Modello logico scelto** (relazionale, gerarchico, a oggetti, documentale, ... → di solito relazionale).

Output

1. **Schema logico** (insieme di tabelle relazionali con chiavi primarie, chiavi esterne, vincoli di integrità).

2. **Documentazione associata**, che descrive le scelte fatte (es. perché una generalizzazione è stata implementata con una tabella unica, oppure perché un attributo multivalore è stato trasformato in una relazione separata).

Passi della Progettazione Logica Relazionale

1. **Rappresentazione delle associazioni $1:1$ e $1:N$**
   - Le associazioni si trasformano in **chiavi esterne**.
   - Nel caso $1:1$ si può scegliere dove mettere la chiave esterna (di solito nella tabella con partecipazione totale).
   - Nel caso $1:N$ la chiave primaria della tabella sul lato "1" diventa chiave esterna nella tabella sul lato "N".

---

2. **Rappresentazione delle associazioni N:M o non binarie**
   - Si crea una **nuova tabella** che contiene come chiavi esterne le chiavi primarie delle entità partecipanti.
   - La combinazione di queste chiavi diventa la **chiave primaria** della tabella di associazione.
   - Se l'associazione ha attributi propri, vengono inseriti come colonne nella nuova tabella.

---

3. **Rappresentazione delle gerarchie di inclusione (generalizzazioni)**

   Tre strategie principali:
   - **Tabella unica (table-per-hierarchy):** un'unica tabella per la superclasse e tutte le sottoclassi; si usa un attributo discriminante.
   - **Tabella per sottoclasse (table-per-subclass):** una tabella per ogni sottoclasse che estende quella della superclasse.
   - **Tabelle separate (table-per-concrete-class):** si eliminano le superclassi e si creano solo tabelle per le sottoclassi, replicando gli attributi comuni.

---

4. **Identificazione delle chiavi primarie**

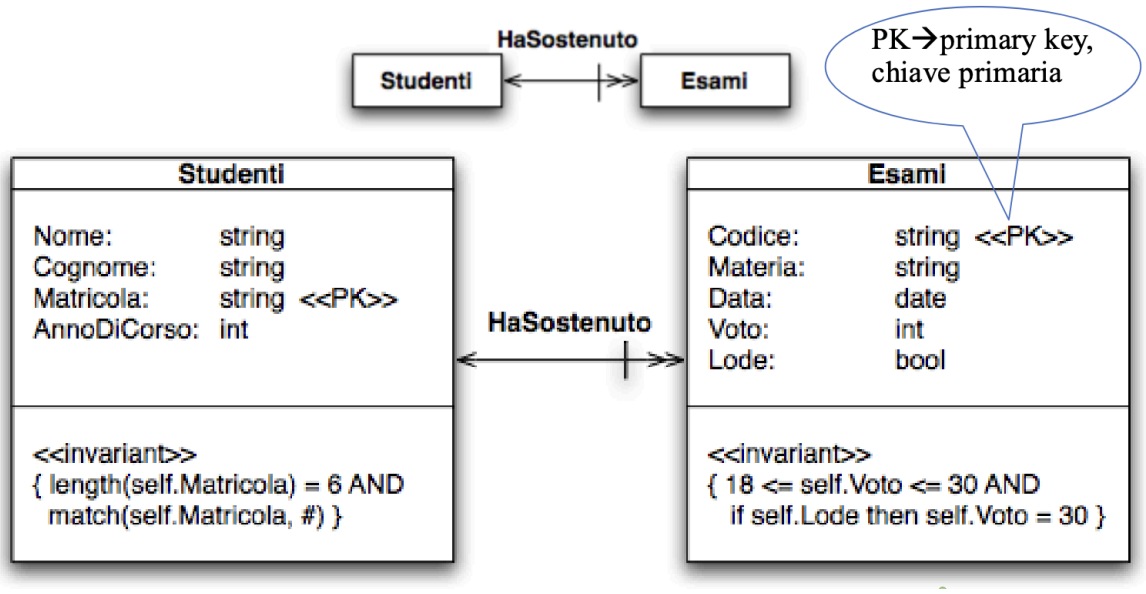
- Ogni tabella deve avere una **chiave primaria**.
- Si eredita la chiave dall'entità corrispondente nel modello concettuale.
- Per le tabelle delle associazioni $N : M$, la chiave primaria è la combinazione delle chiavi esterne.

---

5. **Rappresentazione degli attributi multivalore**
   - Un attributo multivalore diventa una **nuova tabella**.
   - La nuova tabella contiene:
     - la chiave primaria della tabella originale come chiave esterna;
     - l'attributo multivalore come colonna.
   - La chiave primaria è la combinazione di (chiave esterna, valore dell'attributo).

---

6. **Appiattimento degli attributi composti**
   - Un attributo composto si **scompone** nei suoi attributi elementari.
   - Es: `indirizzo (via, numero, città, CAP)` diventa quattro colonne separate nella tabella.



Associazioni $1:N$ (uno a molti)

- **Regola:**

  Nella tabella corrispondente al lato "molti" (N) si aggiunge una **chiave esterna** che punta alla chiave primaria della tabella del lato "uno" (1).

- **Se l'associazione ha attributi propri:**

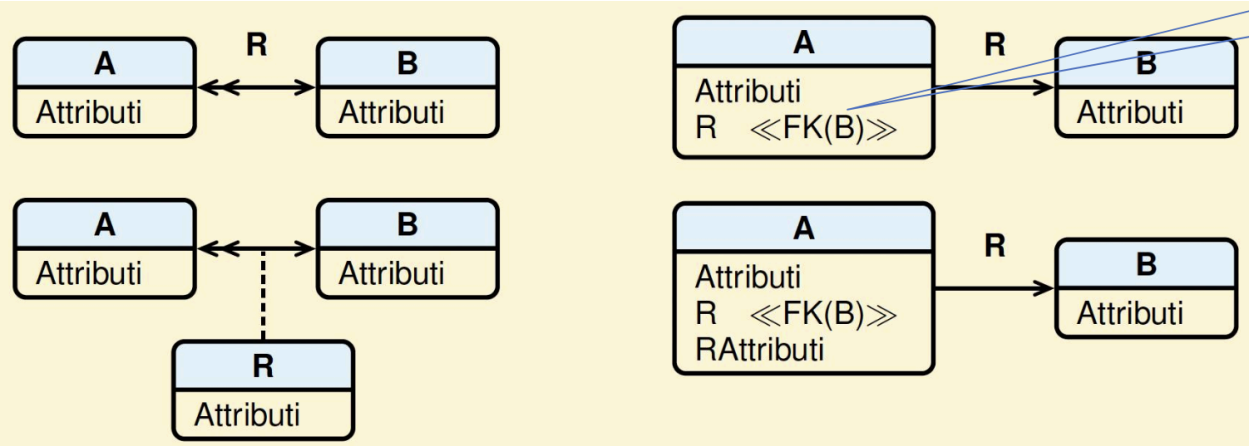
  Gli attributi si aggiungono alla tabella del lato "N" insieme alla chiave esterna.

Esempio concettuale

- Entità: **Dipartimento** (codDip, nome)
- Entità: **Impiegato** (codImp, nome, cognome)
- Associazione: *LavoraIn* (ogni dipartimento ha molti impiegati, ogni impiegato lavora in un dipartimento)

Traduzione logica

- **Dipartimento**(codDip **PK**, nome)
- **Impiegato**(codImp **PK**, nome, cognome, **codDip FK**)



Associazioni $1:1$ (uno a uno)

- **Regola generale:**

  Si aggiunge una **chiave esterna (FK)** in una delle due relazioni, che fa riferimento alla chiave primaria (PK) dell'altra.

- **Criterio di scelta:**
  - Se l'associazione è **totale** da una parte → si mette la FK nell'entità su quel lato.
  - Se non c'è vincolo di totalità → si può scegliere arbitrariamente una delle due tabelle.

- **Se l'associazione ha attributi propri:**

  Gli attributi vengono aggiunti alla tabella dove si inserisce la FK.

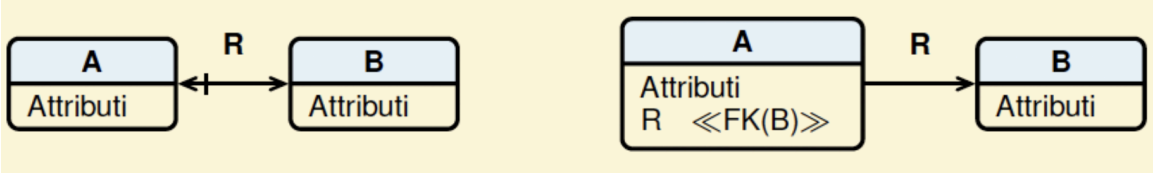
Esempio concettuale

- Entità: **Persona** (codPersona, nome)
- Entità: **Passaporto** (codPass, dataRilascio)
- Associazione: *Ha* (ogni persona ha al massimo un passaporto, ogni passaporto appartiene a una sola persona)

Traduzione logica

- **Persona**(codPersona **PK**, nome, **codPass FK**)
- **Passaporto**(codPass **PK**, dataRilascio)

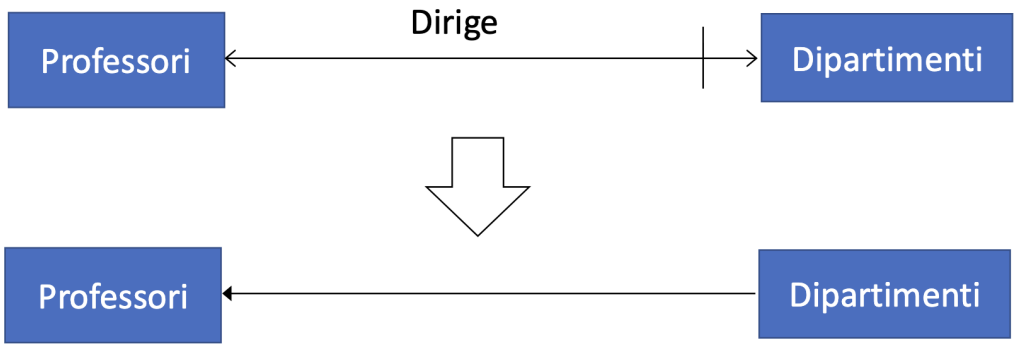
(la FK codPass in Persona indica la relazione $1:1$ con Passaporto)



esempio $(1:1)$

Dirige(Professori, Dipartimenti) ha cardinalità (1:1):

- Un professore può o non può dirigere un solo dipartimento
- Un dipartimento deve avere un (solo) professore come dirigente



Vincoli su Cardinalità ($1:1$ e $1:N$)

1. Direzione dell'associazione

- La **direzione** è la parte in cui mettiamo la **chiave esterna (FK)**.
- Quella parte si dice *"la diretta"* dell'associazione.

---

2. Vincoli principali

- **Univocità della diretta:**

  La FK deve essere coerente con la PK che riferisce (vincolo di integrità referenziale).

- **Totalità della diretta:**

  Se il vincolo è totale (ogni entità della diretta deve partecipare all'associazione), allora la FK deve essere **NOT NULL**.

- **Univocità dell'inversa + Totalità della diretta:**

  In una 1:1, se da entrambi i lati l'associazione è totale e univoca → si applica:

  - **NOT NULL** sulla FK
  - **vincolo UNIQUE** o addirittura **chiave candidata** sulla FK, per garantire che non si ripetano riferimenti.

3. **Esempi**

Caso $1:N$

- Studente(matricola **PK**, nome)
- Corso(codice **PK**, titolo, matricolaStudente **FK**)

Se l'associazione è totale lato corso:

- `matricolaStudente` è **NOT NULL**.

Caso $1:1$

- Persona(codPersona **PK**, nome, codPassaporto **FK UNIQUE NOT NULL**)
- Passaporto(codPassaporto **PK**, dataRilascio)

Qui:

- **NOT NULL** su `codPassaporto` → totalità.
- **UNIQUE** su `codPassaporto` → garantisce che non più persone abbiano lo stesso passaporto.

Associazione 1:1 (Uno a Uno)

Regola generale

- Si inserisce una **chiave esterna (FK)** in una delle due relazioni.
- Se esiste un **vincolo di totalità** da un lato, conviene inserire la FK **nell'entità totale** (perché lì non si rischiano valori NULL).
- Se entrambe sono totali → si può scegliere, ma si aggiunge anche un vincolo di **unicità**.

Esempio 1 – Associazione 1:1 senza vincolo di totalità

Relazione concettuale:

- Persona (codPersona PK, nome)
- Passaporto (codPassaporto PK, dataRilascio)
- Ogni persona può avere al massimo un passaporto, e viceversa.

Implementazione (scelta arbitraria della direzione):

- **Passaporto**(codPassaporto PK, dataRilascio, codPersona FK UNIQUE)

Qui:

- FK `codPersona` assicura il legame con Persona.
- `UNIQUE` garantisce che non ci siano due passaporti per la stessa persona.

Esempio 2 – Associazione 1:1 con vincolo di totalità lato Persona

"Ogni persona deve avere un passaporto."

Implementazione:

- **Persona**(codPersona PK, nome, codPassaporto FK UNIQUE NOT NULL)
- **Passaporto**(codPassaporto PK, dataRilascio)

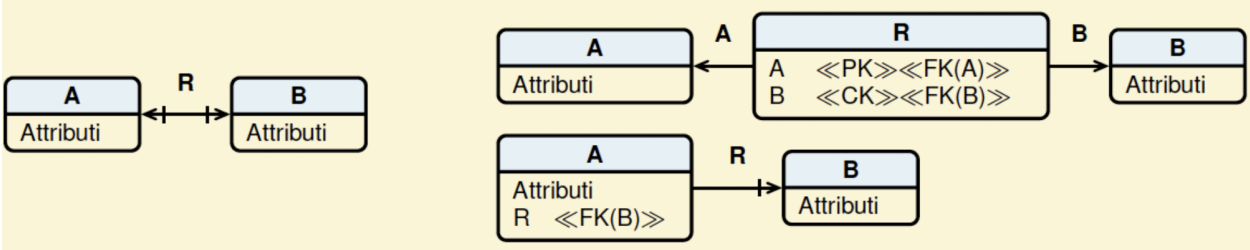
Qui:

- `NOT NULL` su `codPassaporto` perché la totalità impone che ogni persona sia associata a un passaporto.
- `UNIQUE` per mantenere la corrispondenza uno a uno.

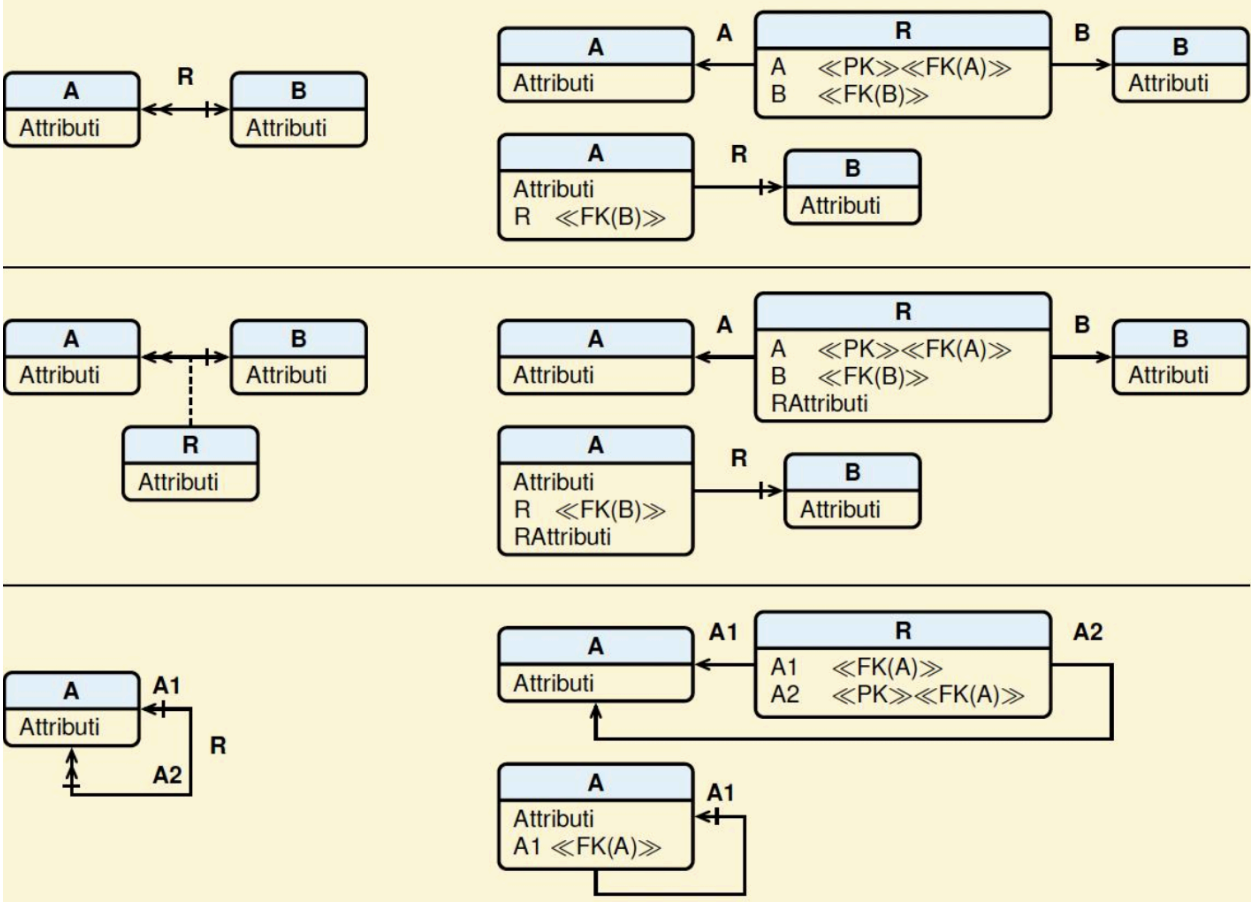
In breve:

- **1:1 senza totalità** → FK da un lato, con UNIQUE.

- **1:1 con totalità** → FK nel lato totale, con UNIQUE + NOT NULL.



Rappresentazione Associazioni Uno a Molti



Rappresentazione delle associazioni molti a molti ($M : N$)

Regola generale

- Si crea una **nuova relazione** (tabella di appoggio).
- Questa nuova relazione contiene:
  - le **chiavi esterne** che riferiscono alle entità coinvolte;
  - eventuali **attributi propri dell'associazione**;
- La **chiave primaria** della nuova relazione è composta da tutte le chiavi esterne (e dagli attributi se necessari).

Esempio 1 – Studenti e Corsi

Relazione concettuale:

- Studente (matricola PK, nome)
- Corso (codCorso PK, titolo)
- Associazione: uno studente può seguire più corsi, e un corso può essere seguito da più studenti.

Implementazione:

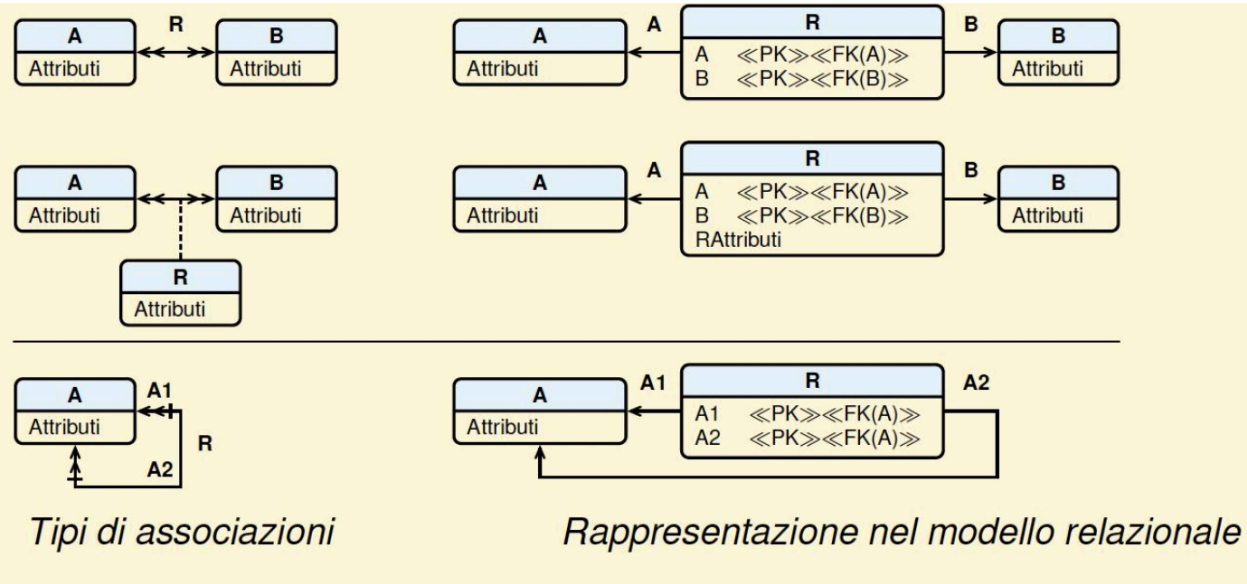
- **Studente**(matricola PK, nome)
- **Corso**(codCorso PK, titolo)
- **Frequenta**(matricola FK, codCorso FK, PRIMARY KEY(matricola, codCorso))

Esempio 2 – Con attributi sull'associazione

"Uno studente segue un corso e ha un voto finale."

- **Frequenta**(matricola FK, codCorso FK, voto, PRIMARY KEY(matricola, codCorso))

Qui l'attributo `voto` appartiene all'associazione, non a Studente né a Corso.


A
Attributi
R
B
Attributi
A
Attributi
R
Attributi
A
Attributi
A
R
A ≪PK≫≪FK(A)≫
B ≪PK≫≪FK(B)≫
B
B
Attributi
A
Attributi
A
R
A ≪PK≫≪FK(A)≫
B ≪PK≫≪FK(B)≫
RAttributi
B
B
Attributi
A
Attributi
A1
R
A2
A
Attributi
A1
R
A1 ≪PK≫≪FK(A)≫
A2 ≪PK≫≪FK(A)≫
A2
Tipi di associazioni
Rappresentazione nel modello relazionale